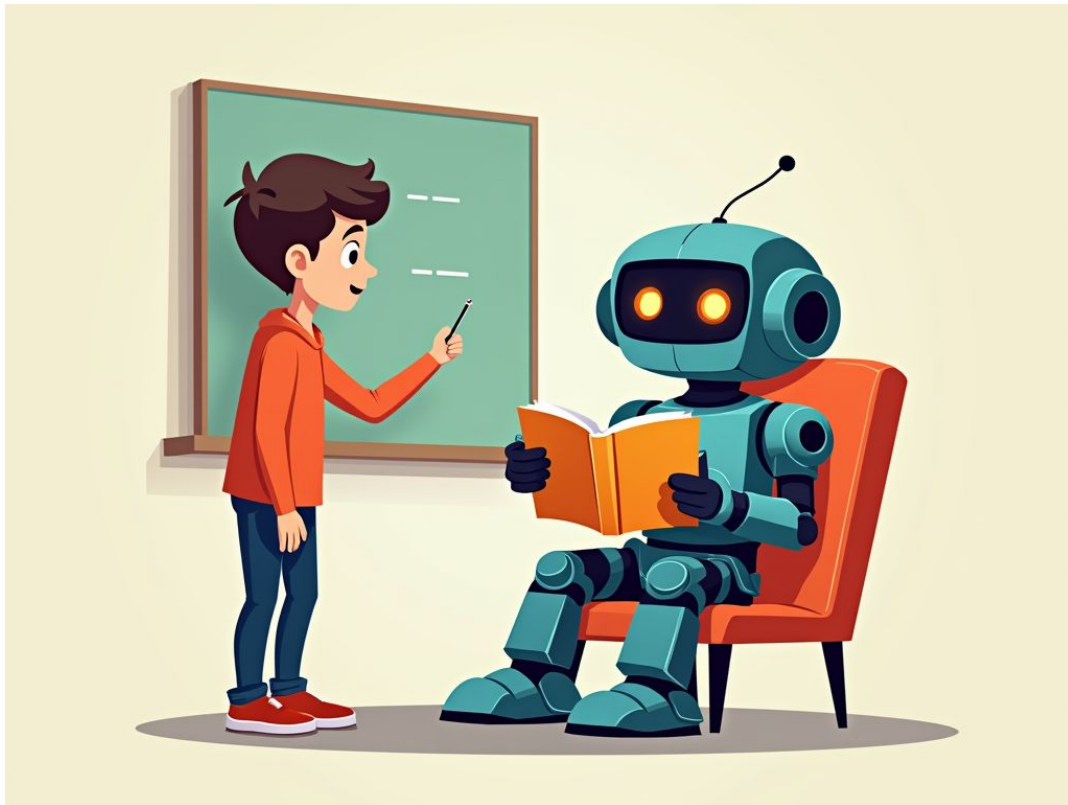


파이썬 기초 프로그래밍과 응용

250220 - 4일차

기계 학습 (Machine Learning)



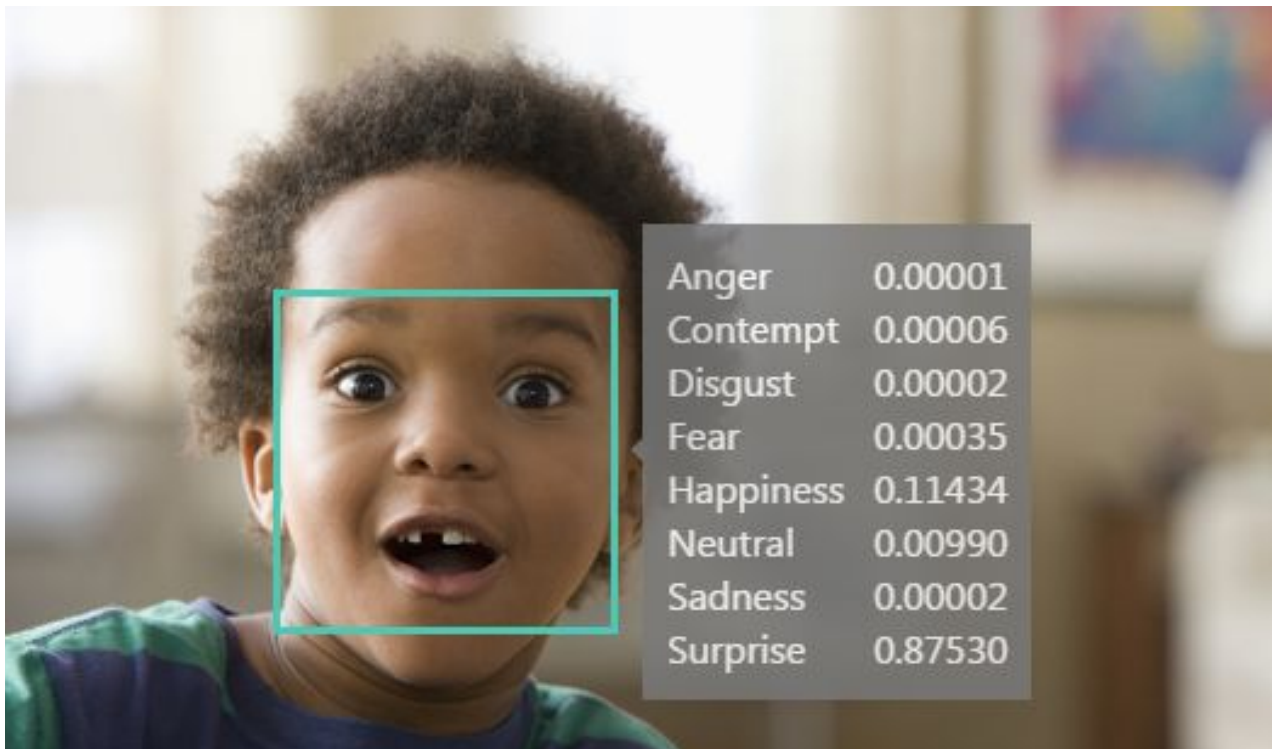
지도 학습 (Supervised Learning)



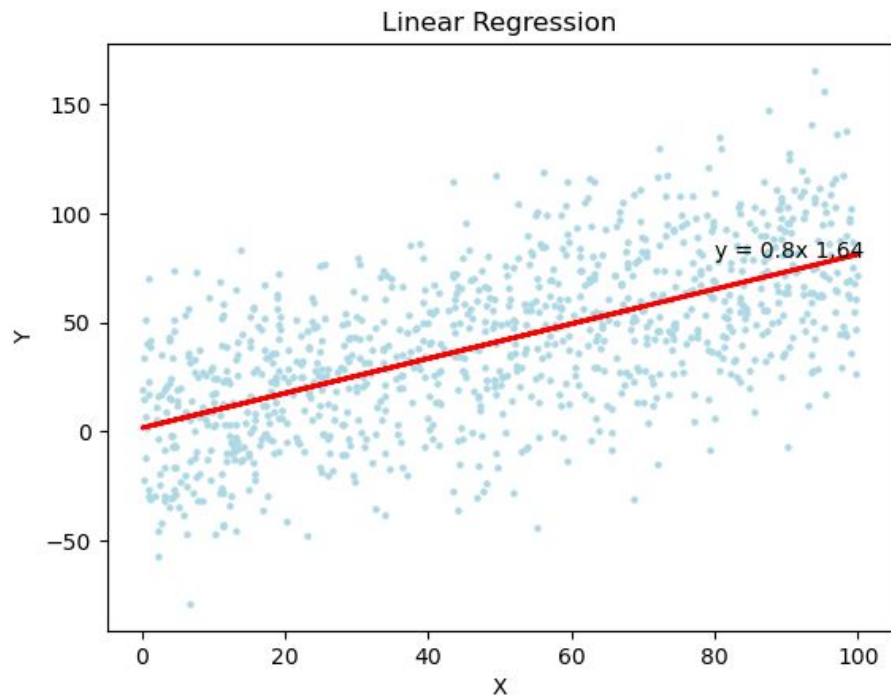
지도 학습 (Supervised Learning)

- 데이터-레이블을 사용하여 모델을 학습
- 새로운 데이터에 대해 예측을 하는 것
 - 분류(Classification): 데이터가 어느 카테고리에 속하는지를 예측
 - 회귀(Regression): 연속적인 값을 예측

지도 학습 (Supervised Learning)



지도 학습 (Supervised Learning)



비지도 학습 (Unsupervised Learning)

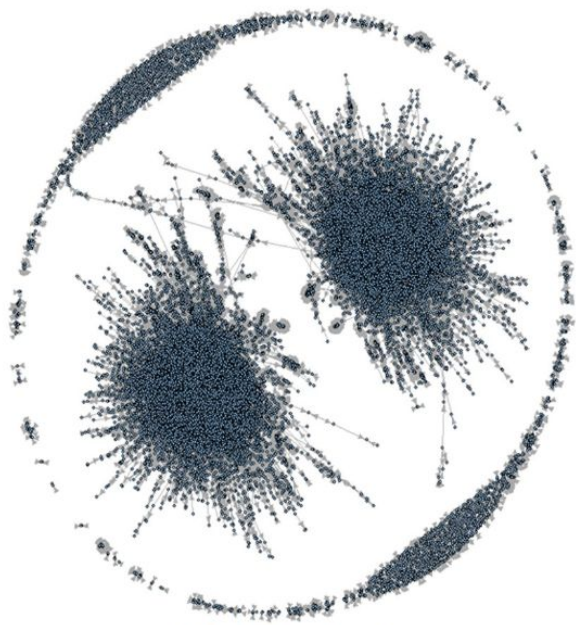


비지도 학습 (Unsupervised Learning)

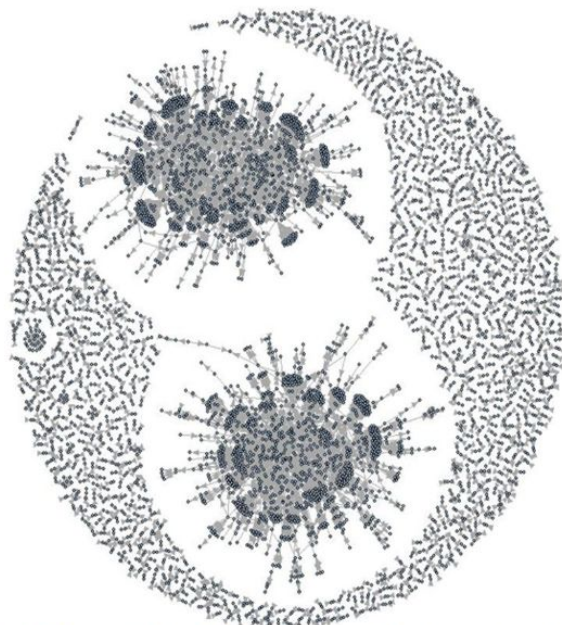
- 데이터만을 사용하여 데이터의 구조나 패턴을 발견
- 데이터의 숨겨진 특징을 탐색하거나 그룹화
 - 군집화(Clustering): 유사한 특성을 가진 데이터 포인트를 그룹화
 - 차원 축소(Dimensionality Reduction): 데이터의 특성 수를 줄여서 분석

비지도 학습 (Unsupervised Learning)

http://ndcreplay.nexon.com/NDC2017/sessions/NDC2017_0013.html



전체 거래 네트워크



대가 없는 게임 머니 및 아이템 이동 네트워크

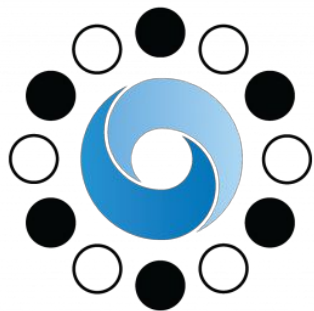
비지도 학습 (Unsupervised Learning)

<https://www.welfare24.net/ab-3101-1905>



강화 학습 (Reinforcement Learning)

- 에이전트가 환경과 상호작용하며 보상을 최대화
 - 알파고

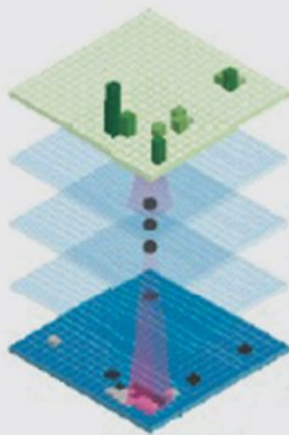


AlphaGo

강화 학습 - 알파고

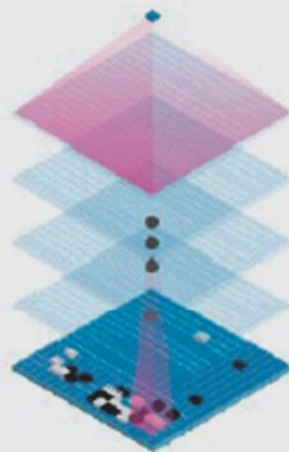
알파고의 정책망과 가치망 구조

정책망



입력된 기보를 바탕으로
특정 상황에 전문가들이 많이 두는
수로 탐색의 폭을 좁혀주는 것

가치망



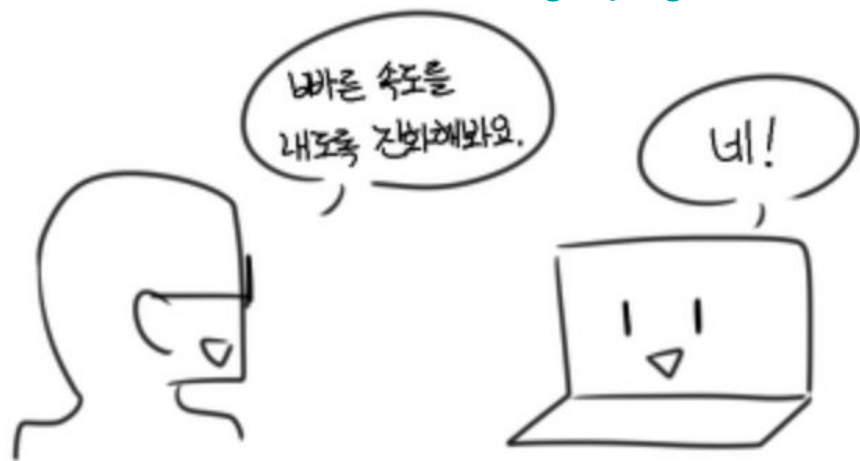
걸러진 수 가운데 현재 대국
상황에서 승산이 높은 수를 예측.
형세를 판단해주는 것

강화 학습 (Reinforcement Learning)

- 상태(State)
 - 에이전트가 현재 처한 상황을 나타내는 정보의 집합
 - 에이전트가 의사결정을 내릴 때 사용하는 환경에 대한 관찰 정보
- 행동(Action)
 - 에이전트가 특정 상태에서 선택할 수 있는 가능한 모든 행위
 - 에이전트의 행동 선택에 따라 환경이 변화하고 새로운 상태로 이동
- 보상(Reward)
 - 에이전트가 특정 행동을 취한 후 환경으로부터 받는 피드백
 - 행동의 좋고 나쁨을 수치화한 평가 지표

강화 학습 실패 사례

https://cgeimage.commutil.kr/phpwas/dgbrdmb_setimgmake.php?w=800&h=-1&m=2&simg=2024012518575779board478478img1.png



<https://youtu.be/NUI6QikjR04>

Q-러닝 알고리즘

- 1989년 크리스토퍼 왓킨스(Christopher Watkins)에 의해 처음 제안
- 세가지 구성요소
 - Q 테이블
 - Q-값(Q-value)
 - 벨만 방정식(Bellman Equation)

Q-러닝 알고리즘

- Q-테이블은 상태(**state**)와 행동(**action**)의 조합에 대한 보상 값(**Q-value**)을 저장
- 행은 상태를, 열은 행동
- 각 셀의 값은 어떤 상태에서 특정 행동을 수행했을 때 기대되는 미래 보상의 총합

Q-러닝 알고리즘

- 탐험(Exploration): 새로운 행동을 시도하여 환경에 대한 지식을 넓히는 것
- 활용(Exploitation): 학습한 정보를 기반으로 최적의 행동을 선택하는 것
- Epsilon-Greedy (ϵ -greedy)
 - 탐험과 활용의 비율을 조절
 - Epsilon 확률(예: 20%)로 무작위 행동을 선택하여 탐험을 수행
 - 1-Epsilon 확률(예: 80%)로 현재 Q-테이블에서 가장 높은 보상이 예상되는 행동을 선택
- 학습이 진행됨에 따라 **epsilon** 값이 점진적으로 감소
 - 초기에는 탐험을 하고 후반에는 학습된 정보를 활용

Q-러닝 알고리즘

- 벨만 방정식(Bellman Equation): Q테이블의 Q값 업데이트를 위한 수식

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- α : 학습률
- γ : 미래 보상에 대해 할인율
- r : 행동 후 받은 보상
- (s, a) : 현재 상태와 현재 행동의 쌍
- $Q(s, a)$: 현재 상태에서 선택 행동을 했을 때 얻을 수 있는 보상 (Q값)
- (s', a') : 다음 상태와 다음 상태에서 가능한 모든 행동의 쌍

Q-러닝 알고리즘

- 벨만 방정식(Bellman Equation): Q테이블의 Q값 업데이트를 위한 수식

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- $\max(Q(s', a'))$: 다음 상태에서 가능한 모든 행동들 중 최대 Q값
- $r + \gamma \max(Q(s', a'))$: 현재 받은 보상과 미래에 받을 수 있는 최대 보상의 합
- $r + \gamma \max(Q(s', a')) - Q(s, a)$: 실제 받은 보상과 예측했던 보상의 차이

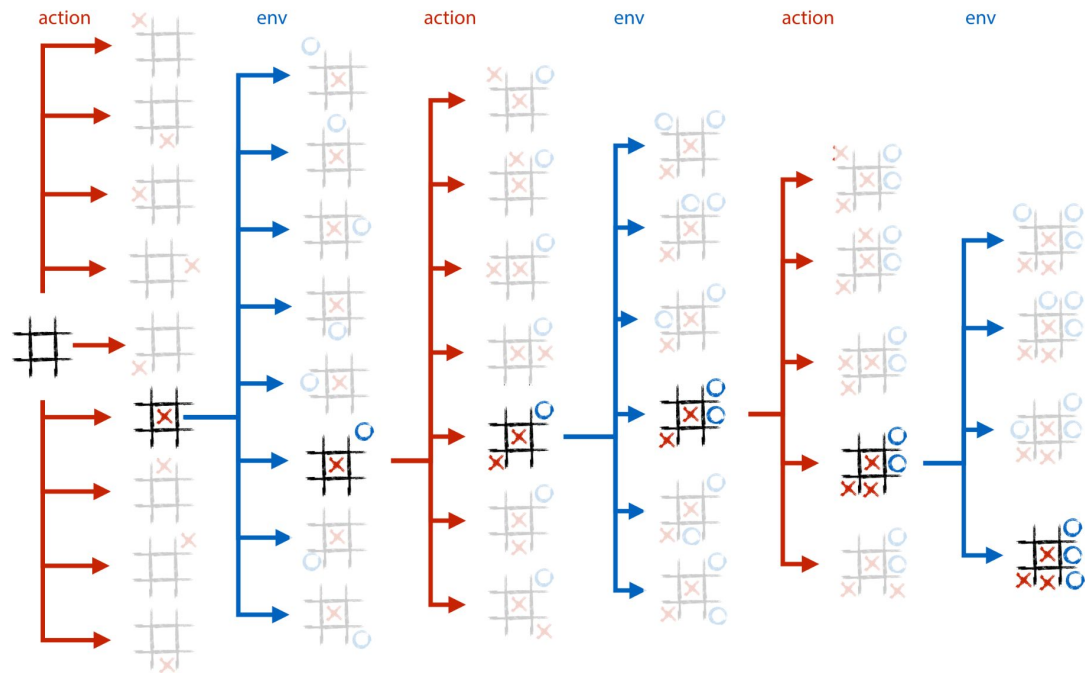
Q-러닝 알고리즘

상태	행동 1	행동 2	행동 3
S1	0.5	0.2	-0.1
S2	0.0	0.8	0.3
S3	-0.3	0.7	0.5

Q-러닝 알고리즘

1. Q-테이블을 0 또는 작은 임의의 값으로 초기화합니다.
2. 현재 상태를 관찰합니다.
3. ϵ -Greedy 정책에 따라 행동 a 를 선택합니다 (탐색 또는 활용).
4. 선택한 행동 a 를 수행하고 보상 r 과 새로운 상태 s' 를 관찰합니다.
5. 벨만 방정식을 사용해 Q-테이블의 Q-값을 업데이트합니다.
6. 3번으로 돌아가 반복합니다.

Q-러닝 알고리즘



$\gamma = 1$

<https://youtu.be/lufECeWtN34>

Jupyter Notebook

- 대화형 개발 환경: 코드, 텍스트, 수식, 시각화를 한 곳에서 실행 가능.
- 다양한 언어 지원: **Python**뿐만 아니라 **R**, **Julia** 등 여러 언어를 지원.
- 확장성: 다양한 확장 기능(예: 자동 완성, 테마 변경 등)을 추가 가능.

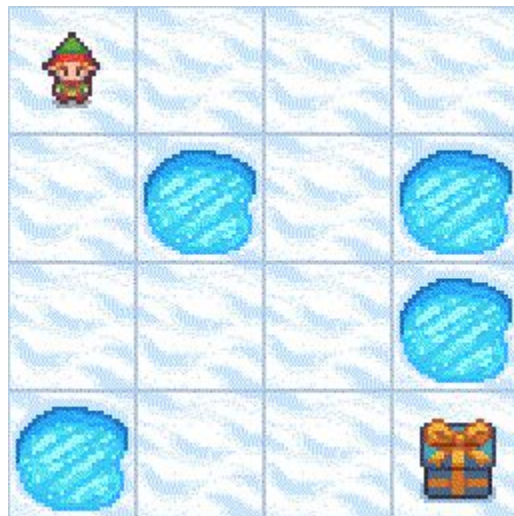
```
pip install notebook
```

```
jupyter notebook
```

Google Colab (<https://colab.research.google.com>)

- 설치 불필요: 웹 브라우저에서 바로 실행 가능.
- 무료 GPU 지원: 강화 학습처럼 계산량이 많은 작업에 유용.
- Jupyter Notebook 호환: Jupyter Notebook 파일(.ipynb)을 그대로 사용 가능.
- 클라우드 기반: 작업 내용이 자동으로 Google Drive에 저장됨.

미로 탈출 에이전트 구현하기



Pandas

- 데이터 분석, 데이터를 효율적으로 조작하고 분석
- 두 가지 데이터 기초 구조
 - **Series**: 1차원 배열 (리스트)
 - **DataFrame**: 2차원 배열 (딕셔너리, 중첩 리스트)

```
pip install pandas
```

Series

1차원 데이터 구조

```
import pandas as pd

# 리스트로부터 Series 생성
s = pd.Series([1, 3, 5, 7, 9])
print(s)
```

DataFrame

- 2차원 데이터 구조
- 다양한 방법으로 생성 가능

```
import pandas as pd

# 딕셔너리로부터 DataFrame 생성
data = {'Name': ['Tom', 'Jerry', 'Mickey'],
        'Age': [20, 21, 19]}
df = pd.DataFrame(data)
print(df)
```

DataFrame

- 2차원 데이터 구조
- 다양한 방법으로 생성 가능

```
import pandas as pd

# 리스트를 사용하여 중첩 배열 생성
nested_list = [[1, 2], [3, 4]], [[5, 6], [7, 8]]

# 중첩 리스트를 DataFrame으로 변환
df_nested = pd.DataFrame(nested_list, columns=['Col1', 'Col2'])

# DataFrame 출력
print(df_nested)
```

Pandas의 데이터 선택 및 필터링

- 열 선택: `df['열이름']`
- 행 선택: `df.loc[인덱스], df.iloc[행번호]`
- 조건 필터링: `df[df['열이름'] 조건]`

```
# 열 선택
ages = df['Age']

# 행 선택
first_row = df.loc[0]

# 조건 필터링
adults = df[df['Age'] > 30]
```

Pandas의 데이터 파일 읽기와 쓰기

```
# CSV 파일로부터 DataFrame 생성  
df = pd.read_csv('data.csv')  
  
# DataFrame을 CSV 파일로 저장  
df.to_csv('output.csv', index=False)
```

Pandas의 데이터 확인

- `head()`: 데이터프레임의 상위 몇 개의 행을 확인합니다.
- `tail()`: 데이터프레임의 하위 몇 개의 행을 확인합니다.
- `info()`: 데이터프레임의 요약 정보를 제공합니다.
- `describe()`: 숫자형 데이터의 통계 요약을 제공합니다.

Pandas의 데이터 확인

```
import pandas as pd

# 데이터프레임 생성
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}
df = pd.DataFrame(data)

# 데이터 확인
print(df.head())
print(df.info())
print(df.describe())
```

Pandas의 데이터 추가 및 삭제

- 열 추가: `df['새열'] = 값`
- 행 삭제: `df.drop(인덱스, axis=0)`
- 열 삭제: `df.drop('열이름', axis=1)`

Pandas의 데이터 추가 및 삭제

열 추가

```
df['City'] = ['New York', 'Los Angeles', 'Chicago']
```

행 삭제

```
df = df.drop(0)
```

열 삭제

```
df = df.drop('City', axis=1)
```

Pandas의 데이터 병합

- `concat()`: 여러 데이터프레임을 연결합니다.
- `merge()`: 두 데이터프레임을 특정 열을 기준으로 병합합니다.

```
# 데이터프레임 병합
df1 = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]})
df2 = pd.DataFrame({'Name': ['Charlie', 'David'], 'Age': [35, 40]})
result = pd.concat([df1, df2])

# 데이터프레임 조인
left = pd.DataFrame({'key': ['A', 'B'], 'value': [1, 2]})
right = pd.DataFrame({'key': ['A', 'B'], 'value': [3, 4]})
merged = pd.merge(left, right, on='key')
```

Pandas의 데이터 시각화

- Matplotlib와 함께 간단한 시각화를 제공

```
import matplotlib.pyplot as plt

# 데이터 시각화
df.plot(kind='bar', x='Name', y='Age')
plt.show()
```

감사합니다